Informe Técnico: Manipulación de Cadenas y Punteros en C/C++

Actividad 4

Ulises Ferrari

Maximo Orue

Lucas Del Pino

Federico Mattia

E.E.S.T. N.° 1 “Eduardo Ader”, Vicente López

Curso: Laboratorio de Programación (LPR), 5.° Año 3.° División

Profesores: Prof. York y Prof. Yamil

Fecha de entrega: 17 de septiembre de 2026

Introducción

El presente informe detalla el desarrollo de la Actividad 4, centrada en el análisis y comprensión de la gestión de memoria de bajo nivel en la arquitectura de un computador, y sigue las normas del estilo APA (7.ª edición). El objetivo principal consistió en implementar un sistema de control de acceso por edad en C++, interactuando de manera directa con los casilleros de la memoria RAM mediante el uso de punteros y operaciones de desreferenciación. El repositorio del proyecto se encuentra en https://github.com/mattia4567/CADENASPUNTEROS.git.

Desarrollo

Modalidad y entorno híbrido (Proyecto Integrador / P.I.A.)

Formato de entrega y modalidad grupal. La actividad se desarrolló bajo la modalidad grupal (Proyecto Integrador - P.I.A.), cumpliendo estrictamente con los estándares formales del formato APA v7 en PDF y asegurando su almacenamiento local bajo la ruta y el nombre obligatorio *Docs/InformeProyectoCADENASPUNTEROS.pdf.*

Integrantes del grupo y distribución de tareas.

- **Ferrari Ulises:** diseño de la estructura del repositorio en Git, configuración del entorno de desarrollo local con VS Code y MinGW, y redacción general de la documentación.
- **Maximo Orue:** análisis de la lógica de punteros, validación de direcciones de memoria en formato hexadecimal y pruebas de consola del código base.
- **Del Pino Lucas:** implementación del código fuente principal (*src/main.cpp*) y resolución de operadores de desreferenciación (* y ->).
- **Mattia Federico:** elaboración del esquema gráfico de la memoria RAM (*Capturas/esquema_ram.png*) y desarrollo de las respuestas teóricas del cuestionario de control.

Entorno de compilación y pruebas. Se empleó un entorno híbrido combinando la terminal integrada de Visual Studio Code con el compilador de MinGW (g++) de forma local, complementado con el control de versiones y repositorio oficial en GitHub estructurado según los requerimientos de la cátedra.

Descripción del código y funcionamiento en la RAM

El programa desarrollado (*src/main.cpp*) declara una variable entera *edad* inicializada en 0 y un puntero de tipo entero *int* p = &edad;*. Mediante el operador de dirección (&), el puntero almacena la ubicación física exacta del casillero de memoria asignado a la variable.

```
int edad = 0;
int* p = &edad;
cin >> *p;
```

Durante la ejecución, la captura de datos se realiza desreferenciando el puntero con *cin >> *p;*, lo que garantiza que el valor ingresado por el usuario se almacene directamente en el espacio de memoria original sin duplicaciones innecesarias.

Cuestionario técnico de control

Pregunta 1: El prototipo y diferencia entre & y *

Explicación. El operador ampersand (*&edad*) extrae y devuelve la dirección física absoluta en la RAM donde se aloja la variable, mientras que el asterisco acoplado al puntero (**p*) efectúa una desreferenciación para acceder al contenido almacenado en el interior de dicho casillero de memoria.

Respuesta. La instrucción *cout << p;* imprime en la consola la dirección de memoria hexadecimal física (por ejemplo, 0x61fe1c), indicando el casillero exacto de la RAM. En contraparte, la instrucción *cout << *p;* imprime el valor numérico entero que se encuentra guardado dentro de ese casillero (en este caso, la edad ingresada por el usuario).

Pregunta 2: Error crítico al guardar entrada

Explicación. Omitir el operador de desreferenciación al intentar capturar datos mediante la entrada estándar (*cin >> p;*) genera un conflicto tipográfico y de seguridad en el

sistema.

Respuesta. Es estrictamente obligatorio desreferenciar el puntero utilizando **p* porque, de lo contrario, el compilador intentaría sobreescribir la dirección de memoria hexadecimal que el puntero almacena en lugar de modificar el valor de la variable original. Esto ocasiona un error crítico de compilación o fallas graves de segmentación en tiempo de ejecución.

Pregunta 3: Formato hexadecimal en la memoria

Explicación. Las direcciones físicas de la memoria RAM se gestionan a nivel de hardware utilizando bases numéricas óptimas para la representación binaria agrupada.

Respuesta. El sistema operativo emplea la notación hexadecimal (base 16, identificada con el prefijo 0x) porque permite compactar y simplificar la lectura de secuencias binarias extensas. Facilita enormemente el trabajo de los programadores y depuradores al representar de manera limpia y estandarizada los bytes de la memoria en comparación con el sistema decimal tradicional.

Conclusión

La actividad permitió comprender cómo un programa en C++ accede a la memoria RAM mediante punteros: el operador de dirección (&) obtiene la ubicación de una variable y el operador de desreferenciación (*) permite leer o modificar su contenido. Se aplicó este concepto en un sistema de control de acceso por edad y se comprendió por qué las direcciones de memoria se representan en notación hexadecimal. El trabajo en equipo, el entorno con VS Code y MinGW y el versionado en GitHub completaron el proceso de documentación del proyecto.

Referencias

Ferrari, U., Orue, M., Del Pino, L., y Mattia, F. (2026). *CADENASPUNTEROS* [Repositorio de código]. GitHub. https://github.com/mattia4567/CADENASPUNTEROS.git